

Задание 1

Код который я написал:

```
a = 42
b = 3.14159
c = "Hello, Python!"
d = True
e = [1, 2, 3]
f = (4, 5, 6)
g = {"name": "Alice", "age": 30}
h = {7, 8, 9}
i = None

print('Тип переменной a:', type(a))
print('Тип переменной b:', type(b))
print('Тип переменной c:', type(c))
print('Тип переменной d:', type(d))
print('Тип переменной e:', type(e))
print('Тип переменной f:', type(f))
print('Тип переменной g:', type(g))
print('Тип переменной h:', type(h))
print('Тип переменной i:', type(i))
```

Компиляция кода:

```
Тип переменной a: <class 'int'>
Тип переменной b: <class 'float'>
Тип переменной c: <class 'str'>
Тип переменной d: <class 'bool'>
Тип переменной e: <class 'list'>
Тип переменной f: <class 'tuple'>
Тип переменной g: <class 'dict'>
Тип переменной h: <class 'set'>
Тип переменной i: <class 'NoneType'>
```

Задание 2

1)Код который я написал с комментарием:

```
my_list=[1, 2, 3]
print('Исходный список:', my_list)
my_list[0]=100
print('Измененный список:', my_list)
#Поменял первый элемент списка
```

Компиляция кода:

```
Исходный список: [1, 2, 3]
Измененный список: [100, 2, 3]
```

2)Код который я написал с комментарием:

```
my_tuple=(1, 2, 3)
print('Исходный список:', my_tuple)
my_tuple[0]=100
print('Попытка изменить список:', my_tuple)
#Попытался поменять кортеж, т.к. он неизменяемый ничего не получилось
```

Компиляция кода:

```
Traceback (most recent call last):
  File "11568599.py", line 3, in <module>
    my_tuple[0]=100
    ~~~~~^
TypeError: 'tuple' object does not support item assignment
Исходный список: (1, 2, 3)
```

3)Код который я написал с комментарием:

```
my_string='cat'
print('Изначальная строка:', my_string)
my_string[0]='b'
print('Измененная строка:', my_string)
#Попытка изменить строку тоже не увенчалась успехом, строка тоже является неизменяемой переменной
```

Компиляция кода:

```
Traceback (most recent call last):
  File "11568660.py", line 3, in <module>
    my_string[0]='b'
    ~~~~~^
TypeError: 'str' object does not support item assignment
Изначальная строка: cat
```

Задание 3

Изначальный код без try-except:

```
num1=input()
num2=input()
num2=int(num2)
num1=int(num1)
print(num1+num2)
```

Компиляция с числами 2 и 6:

8

Компиляция с числами 2 и 5.5:

```
Traceback (most recent call last):
  File "11568722.py", line 3, in <module>
    num2=int(num2)
    ^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: '5.5'
(программа ничего не вывела)
```

Код с try-except:

```
try:
    num1=input()
    num2=input()
    num2=int(num2)
    num1=int(num1)
    print(num1+num2)
except:
    print('Вы ввели некорректные данные для этой программы.')
```

Компиляция с числами 2 и 6:

8

Компиляция с числами 2 и 5.5:

Вы ввели некорректные данные для этой программы.

Задание 4

Для “Хранение текущего количества студентов в группе.” я выберу переменную `int()`, потому что она хранит целочисленные числа

Для “Хранение температуры в градусах Цельсия (например, 23.5)” я выберу переменную `float()`, потому что она хранит числа с цифрами после запятой

Для “Хранение логина пользователя (например, "ivanov_ivan")” я выберу переменную `str()`, потому что она хранит информацию в виде строки.

Для “Хранение информации о том, включено ли уведомление (да/нет)” я выберу переменную `bool()`, потому что она имеет два состояния: либо `True`, либо `False`

Для “Хранение списка любимых фильмов” я выберу переменную `list()`, потому что фильмов может быть много и список надо дополнять

Для “Хранение координат точки в 2D пространстве (x, y), которые не должны изменяться после задания” я выберу переменную `tuple()`, потому что кортеж неизменяемый

Для “Хранение уникальных ID товаров в корзине покупок” я выберу переменную `set()`, потому что она автоматически исключает повторения

Для “Хранение соответствия между названием страны и её столицей” я выберу переменную `dict()`, потому что словарь может связать два значения вместе

Задание 5

Код который я написал:

```
shopping_list=['молоко', 'сосиски', 'огурцы']
shopping_list.append('помидоры')
print('Список покупок:', shopping_list)
unique_items=set(shopping_list)
print('Список покупок:', unique_items)
```

Компиляция кода:

```
Список покупок: ['молоко', 'сосиски', 'огурцы', 'помидоры']
Список покупок: {'сосиски', 'огурцы', 'молоко', 'помидоры'}
```

set() и list() различаются тем, что set() не хранит дубликаты и порядок в переменное в отличии от list()

Задание 6

Код который я написал:

```
name = input('Ваше имя: ')
age_str = input('Ваш возраст: ')
subjects_str = input('Любимые предметы (через запятую): ')
age = int(age_str)
subjects_list = [subject.strip() for subject in subjects_str.split(",")]
student = {
    'name': name,
    'age': age,
    'subjects': subjects_list
}
print('=' * 30)
print('АНКЕТА СТУДЕНТА')
print('=' * 30)
print('Имя:', student['name'])
print('Возраст:', student['age'])
print('Любимые предметы:', student['subjects'])
print('=' * 30)
```

Компиляция кода:

```
АНКЕТА СТУДЕНТА
=====
Имя: Арсений
Возраст: 18
Любимые предметы: ['Математика', 'физика', 'информатика']
=====
```

1.Какие типы данных являются простыми (скалярными)?

К простым типам относятся те, которые содержат в себе только одно значение: целые числа `int()` числа с плавающей точкой `float()`, логические значения `bool()` и строки `str()`.

2.В чём разница между `list()` и `tuple()`?

Главное отличие — изменяемость. Список `list()`— изменяемый тип (динамический массив, элементы можно удалять и менять). Кортеж `tuple()`— неизменяемый тип (статический массив, после создания его изменить нельзя).

3.Почему важно понимать, является ли тип данных изменяемым или нет?

Это критично для безопасности данных (чтобы функции случайно не изменили исходный объект) и для правильного написания кода. Например, ключами в словарях `dict()` или элементами множеств `set()` могут быть только неизменяемые типы (числа, строки, кортежи).

4.Зачем нужно преобразование типов? В каких случаях оно обязательно?

Преобразование нужно, чтобы привести данные к формату, с которым можно выполнять требуемые операции. Оно обязательно при обработке пользовательского ввода через `input()`, так как эта функция всегда возвращает строку `str()`. Чтобы выполнить с вводом математические операции, строку обязательно нужно перевести в `int()` или `float()`.

5.Какие ошибки могут возникнуть при преобразовании, и как с ними бороться?

Чаще всего возникает ошибка `ValueError` (когда данные невозможно перевести в указанный тип, например `int('a,b,c')` или `TypeError`. С ними борются с помощью конструкции обработки исключений `try-except`.